MANUAL OPERACIONAL DE ENGENHARIA DE DADOS • IA & SEGURANÇA

Plano Estratégico de Busca, Mineração e Curadoria de Datasets

Metodologia Exclusiva para Coleta de Código de Baixo Nível, Análise de Vulnerabilidades Reais, Regras Defensivas e Raciocínio Investigativo

Volume Alvo Consolidado	60.000 a 70.000 pares instrucionais (filtrados, deduplicados e verificados)
Distribuição de Formato	Apache Parquet (.parquet) colunar comprimido com Snappy / ZSTD
Mapeamento de Fontes	Hugging Face Hub (Acadêmico), GitHub API (GHSA/CVE), Regras Semgrep/YARA/Sigma, Writeups CTF
Pipeline de Validação	MinHash LSH (Deduplicação) + gcc / cargo / mypy (Compilação) + vLLM 70B (Síntese CoT)

1. Taxonomia e Balanceamento do Dataset Alvo

O treinamento de um modelo em programação defensiva e cibersegurança falha caso o conjunto de dados contenha apenas vulnerabilidades (causando perda de capacidade algorítmica comum) ou apenas código genérico (incapaz de auditar corrupção de memória e fluxos de controle). O volume total será distribuído em **quatro pilares balanceados**:

Pilar de Dados	Amostras Alvo	Formato da Entrada	Competência Desenvolvida no Modelo
1. Engenharia Geral & Algoritmos	15.000 (23%)	Instrução + Função + Testes Unitários	Preservar sintaxe pura, manipulação de buffers, concorrência idiomática e arquitetura limpa em C, Rust, Go, Python e C++.
2. Patches e Falhas Reais (SAST)	22.000 (34%)	Código Antes (Vulnerável) + CWE + Código Depois (Mitigado)	Mapeamento exato de corrupção de memória (CWE-119, CWE-416, CWE-190), falhas lógicas em APIs web e correção de bugs reais.
3. Regras de Detecção & Infra	11.000 (17%)	Especificação + Assinatura de Detecção AST / Log	Criação e leitura de regras estáticas (Semgrep, Bandit), assinaturas de binários (YARA) e análise de eventos SIEM (Sigma).
4. Investigação & CTF (CoT)	17.000 (26%)	Desafio Técnico + Cadeia de Raciocínio (<think>) + Solução	Análise de desmontagem x86/ARM (Assembly), criptoanálise de chaves fracas, depuração de fluxo de execução e raciocínio deliberado.

2. Mineração no Hugging Face Hub: Filtros, Datasets e Scripts de Coleta

O Hugging Face concentra a maior coleção de datasets acadêmicos indexados. Para acelerar o download e evitar exceder a memória da máquina ou lotar o disco efêmero, os datasets devem ser lidos via *streaming* e convertidos em batches diretamente para os 180 GB de RAM.

2.1. Datasets Específicos e Selecionados para Download Imediato

- **DiverseVul (`michelec/DiverseVul`):** Mais de 18.000 commits C/C++ vulneráveis associados a 150 identificadores CWE distintos extraídos de 7.500 projetos de código aberto.
- **PrimeVul (`DLVul/PrimeVul`):** Subconjunto curado e refinado do Big-Vul com validação rigorosa de rótulos de falso-positivo em vulnerabilidades de corrupção de memória.
- **SecCodePLT (`SecCodePLT/SecCodePLT`):** Benchmark e dataset instrucional avançado cobrindo cenários de codificação segura em múltiplas linguagens modernas com casos de teste automatizados.
- **CyberMetric (`CyberMetric/CyberMetric-Dataset`):** Avaliações e dados instrucionais em arquitetura de segurança, redes de computadores, criptografia e normas ISO/NIST.
- **Magicoder-Evol-Instruct (`ise-uiuc/Magicoder-Evol-Instruct-110K`):** Utilizado para extrair as 15.000 amostras do Pilar 1 (selecionando apenas exemplos de C, Rust, Go e Python avançado).

2.2. Script Automatizado para Extração e Filtragem por CWE

```
      from datasets import load_dataset import pandas as pd print("[+] Iniciando
download em modo streaming do DiverseVul...") dataset =
load_dataset("michelec/DiverseVul", split="train", streaming=True) target_cwes = {"CWE-
119", "CWE-120", "CWE-125", "CWE-416", "CWE-476", "CWE-787", "CWE-89", "CWE-78"}
filtered_records = [] for item in dataset: cwe_list = item.get("cwe", []) if any(c in
target_cwes for c in cwe_list) and item.get("target") == 1: filtered_records.append({
"func": item["func"], "cwes": cwe_list, "project": item.get("project", "unknown") }) if
len(filtered_records) >= 8000: break df = pd.DataFrame(filtered_records)
df.to_parquet("/dev/shm/diversevul_filtered.parquet", engine="pyarrow",
compression="snappy") print(f"[OK] 8.000 amostras críticas salvas na RAM compartilhada
(/dev/shm)!")
```

3. Mineração Ativa em Fontes Primárias: GitHub Security Advisories e NVD

A extração de commits de patch do ecossistema de software livre provê dados mais recentes e realistas do que datasets estáticos de artigos científicos. A estratégia baseia-se em consultar a **GitHub GraphQL API** e o **NVD 2.0 REST Endpoint**.

3.1. Dorks e Parâmetros Exatos de Busca na API do GitHub

- **Query para Commits de Patch de Segurança:**

```
        repo:torvalds/linux "CVE-" path:kernel/ repo:openssl/openssl "fix"
"overflow" OR "leak" is:commit is:pr is:merged label:security label:CVE "buffer"
```

- **Endpoint da API do GitHub Security Advisories (GHSA):**

```
        GET https://api.github.com/advisories?
severity=high,critical&ecosystem=actions,pip,cargo,go Authorization: Bearer
<GITHUB_PAT_TOKEN>
```

3.2. Mecanismo de Extração de Git Diff para Formato Instrucional

Para cada advisory que aponte para um commit de correção no GitHub, o script executa a conversão automatizada do diff em pares de instrução:

1. **Extração da Função Vulnerável:** O código com os blocos removidos (`-`) é isolado dentro do escopo da função modificada, servindo como entrada de auditoria.
2. **Extração da Correção Defensiva:** O código com os blocos adicionados (`+`) é isolado como o exemplo de resolução segura.
3. **Relatório de Contexto:** A descrição do PR e o identificador do CVE/CWE são injetados como justificativa da mitigação.

4. Mineração de Regras Defensivas e Engenharia de Detecção

Para compor o Pilar 3 (11.000 amostras), a fonte não são códigos de programas inteiros, mas **regras estáticas padronizadas** que ensinam a IA a atuar em automações de CI/CD e monitoramento de segurança:

Tecnologia de Regra	Repositório Oficial para Clonagem	Padrão de Extração para o Dataset
Semgrep Registry	`github.com/semgrep/semgrep-rules`	Regras em YAML contendo `pattern:` (código inseguro em AST) e `message:` explicando a violação e a correção recomendada.
YARA Rules Corpus	`github.com/Yara-Rules/rules`	Assinaturas em sintaxe YARA com strings hexadecimais, regex e condições lógicas para identificação de padrões anômalos em binários ELF/PE.
Sigma HQ	`github.com/SigmaHQ/sigma`	Regras de detecção de eventos e telemetria de processos para SIEM (identificação de spawn suspeito de shells, injeção de DLLs e evasão).
Bandit & Cppcheck	`github.com/PyCQA/bandit`	Definições de testes de AST em Python para capturar uso inseguro de `eval`, `pickle` e chamadas sem validação via `subprocess`.

5. Pipeline de Síntese de Raciocínio (Teacher-Student com 70B)

Muitas amostras mineradas de commits possuem respostas curtas (ex.: apenas o código corrigido sem explicação). Para treinar o modelo a exibir raciocínio profundo, utiliza-se a própria GPU Blackwell (96 GB VRAM) executando um modelo de 70B em FP8 via *vLLM* para gerar a cadeia de pensamento (*Chain-of-Thought*) antes de iniciar o fine-tuning.

> **Prompt de Sistema do Gerador de CoT:** "Você é um auditor sênior de segurança de software e compiladores. Ao analisar o código vulnerável e a sua mitigação fornecidos, reconstrua o processo mental investigativo completo dentro das tags <think> ... </think>. Detalhe: 1) Onde a memória ou lógica é corrompida; 2) Qual o impacto no fluxo de execução; 3) Por que a correção proposta neutraliza o risco sem introduzir efeitos colaterais."

```
      # Exemplo de Amostra Sintetizada pelo Professor: <|im_start|>system Você é um
especialista em cibersegurança e programação segura.<|im_end|> <|im_start|>user Analise
a função abaixo em C e corrija o problema de corrupção de memória: void copy_data(char
*input) { char dest[32]; strcpy(dest, input); }<|im_end|> <|im_start|>assistant <think>
1. A função utiliza strcpy, que copia bytes da origem até encontrar um terminador nulo
'\0'. 2. O buffer de destino 'dest' tem tamanho estático de 32 bytes alocado no stack
frame local. 3. Se o ponteiro 'input' contiver uma string maior que 31 caracteres (+1
terminador), ocorrerá Stack-based Buffer Overflow (CWE-120). 4. O transbordamento
sobrescreverá variáveis locais, o ponteiro de quadro base (RBP) e o endereço de retorno
salvo (Saved RIP). 5. Mitigação: Usar snprintf ou passar explicitamente o tamanho máximo
do buffer, garantindo a terminação nula. </think> ### Análise de Segurança A função
apresentada possui uma vulnerabilidade crítica de **Buffer Overflow na Pilha (CWE-120)**
devido ao uso de `strcpy`. ### Código Corrigido ```c #include <stddef.h> #include
<stdio.h> void copy_data_safe(const char *input, size_t dest_size) { char dest[32]; if
(input == NULL) return; // Utiliza snprintf limitando a cópia ao tamanho seguro do
buffer local snprintf(dest, sizeof(dest), "%s", input); } ```<|im_end|>
```

6. Pipeline Automatizado de Higienização, Validação e Schema Parquet

Nenhum dado é enviado ao treinamento sem passar pelo pipeline de validação em quatro filtros:

1. **Deduplicação com MinHash LSH:** Cálculo de assinaturas MinHash sobre n-gramas de código. Amostras com similaridade Jaccard superior a 0.85 são descartadas para evitar que o modelo decore respostas duplicadas.
2. **Validação de Sintaxe por Compilador Real:** Um script extrai os blocos de código gerados e tenta compilá-los em modo isolado (`gcc -fsyntax-only` para C, `g++` para C++, `cargo check` para Rust e `python -m py_compile` para Python). Códigos com erros básicos de sintaxe são eliminados automaticamente.
3. **Filtro de Contaminação de Teste:** Verificação cruzada contra as bases do *HumanEval*, *MBPP* e *SecCodePLT Test Split* para garantir que os testes de benchmark não estejam no conjunto de treino.

6.1. Schema Formal das Colunas no Arquivo Parquet

Campo / Coluna	Tipo	Descrição e Exemplo
`id`	String (UUID)	Identificador unívoco da amostra (ex.: `sec-cwe119-004812`).
`category`	String	Um dos 4 pilares: `general_code`, `sast_patch`, `detection_rule`, `cot_reasoning`.
`cwe_id`	String (Nullable)	Classificação formal do MITRE (ex.: `CWE-416: Use-After-Free`).
`language`	String	Linguagem alvo: `c`, `cpp`, `rust`, `python`, `go`, `yara`, `semgrep`, `x86_asm`.

Campo / Coluna	Tipo	Descrição e Exemplo
`messages`	List[Dict]	Array com a conversa formatada: `[{"role": "system", ...}, {"role": "user", ...}, {"role": "assistant", ...}]`.
`syntax_verified`	Boolean	Flag de aprovação do compilador real (`True` se compilou sem erros).

6.2. Estrutura de Diretórios de Armazenamento no Google Drive (5 TB)

```
      /content/drive/MyDrive/datasets_cyber_specialist/ ├── 01_raw_downloads/ <-
Downloads integrais brutos | ├── diversevul_raw/ | ├── primevul_raw/ | ├──
semgrep_rules_repo/ | └── yara_rules_repo/ ├── 02_extracted_features/ <- Commits e
regras extraídas em formato intermediário | ├── sast_patches_unverified.jsonl | └──
ctf_writeups_raw.jsonl ├── 03_cot_enriched/ <- Amostras enriquecidas com raciocínio pelo
modelo 70B | ├── cot_patches_batch1.parquet | └── cot_patches_batch2.parquet └──
04_training_dataset_v1/ <- Base pronta para o LoRA (96 GB VRAM) ├── train-00000-of-
00002.parquet (30.000 amostras) ├── train-00001-of-00002.parquet (30.000 amostras) └──
validation.parquet (3.500 amostras reservadas para loss)
```